



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto *Query Analyzer*

Curso: *Base de Datos II*

Docente: *Patrick Cuadros Quiroga*

Integrantes:

Carbajal Vargas, Andre Alejandro (2023077287)

Yupa Gómez, Fátima Sofía (2023076618)

Tacna - Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-04	Versión inicial
1.1	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-06-23	Actualización factual y formato institucional
1.2	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-07-04	Ampliación del estudio de factibilidad, costos y análisis financiero

ÍNDICE GENERAL

1. [Descripción del Proyecto](#)
2. [Nombre del proyecto](#)
3. [Duración del proyecto](#)
4. [Descripción](#)
5. [Objetivos](#)
6. [Riesgos](#)
7. [Análisis de la Situación Actual](#)
8. [Planteamiento del problema](#)
9. [Consideraciones de hardware y software](#)
10. [Estudio de Factibilidad](#)
11. [Factibilidad Técnica](#)
12. [Factibilidad Económica](#)
13. [Factibilidad Operativa](#)
14. [Factibilidad Legal](#)
15. [Factibilidad Social](#)
16. [Factibilidad Ambiental](#)
17. [Análisis Financiero](#)
18. [Justificación de la inversión](#)
19. [Criterios de inversión](#)
20. [Conclusiones](#)

Informe de Factibilidad

1. Descripción del Proyecto

1.1. Nombre del proyecto

El proyecto se denomina **Query Analyzer** y corresponde al sistema **Analizador de Rendimiento de Consultas**.

1.2. Duración del proyecto

El desarrollo académico se inició el **4 de abril de 2026** y se consolidó durante los meses de abril, mayo y junio de 2026. La versión documentada en este informe corresponde a la línea funcional 2.3.x, que incorpora CLI, TUI, API REST, servidor MCP, extensión para Visual Studio Code, automatización de calidad y publicación de documentación.

Fase	Periodo aproximado	Resultado principal
Inicio y arquitectura	Abril 2026	Estructura modular, modelos base y registro de adaptadores
Adaptadores y perfiles	Abril - mayo 2026	Soporte inicial para motores SQL, NoSQL, grafos y series de tiempo
Interfaces de usuario	Abril - mayo 2026	CLI, TUI, historial local y exportación de reportes
API, IA e integraciones	Junio 2026	API REST, análisis IA opcional, MCP y extensión VS Code
Calidad y documentación	Junio - julio 2026	Evidencias, Pages, informes académicos y actualización de FD01

1.3. Descripción

Query Analyzer es una herramienta local, multiplataforma y extensible que permite obtener planes de ejecución, métricas observables y evidencia técnica de consultas en múltiples motores de bases de datos. El sistema busca resolver la fragmentación que aparece cuando cada motor ofrece comandos, formatos y métricas diferentes para analizar el rendimiento.

El proyecto se desenvuelve en el contexto del curso **Base de Datos II** y atiende una necesidad académica y práctica: facilitar el estudio de planes de ejecución sin depender de una herramienta distinta por cada motor. Para ello, la solución implementa un patrón Adapter sobre motores SQL, NoSQL, grafos, series de tiempo y servicios cloud. El usuario puede interactuar mediante CLI, TUI, API REST local, servidor MCP o extensión de Visual Studio Code.

El principio central del sistema es la **separación entre evidencia factual e interpretación**. El núcleo reporta datos reales obtenidos del motor o derivados de su estructura de plan. La interpretación mediante inteligencia artificial es opcional y se presenta aparte, sin modificar las métricas ni generar puntuaciones universales.

1.4. Objetivos

1.4.1. Objetivo general

Desarrollar una herramienta multiplataforma y extensible que permita analizar planes de ejecución y métricas observables de consultas en múltiples motores de bases de datos, presentando la evidencia mediante interfaces uniformes y manteniendo separada cualquier interpretación opcional generada por inteligencia artificial.

1.4.2. Objetivos específicos

Objetivo específico	Logro esperado
Diseñar una arquitectura basada en capas y patrón Adapter	Permitir que cada motor tenga una implementación propia sin cambiar el flujo principal
Implementar un contrato común de análisis	Construir reportes con motor, consulta, tiempo observado, plan normalizado, plan original y métricas disponibles
Gestionar perfiles locales de conexión	Crear, listar, probar, seleccionar y eliminar perfiles con cifrado de credenciales
Diagnosticar conexiones	Verificar configuración, conectividad, autenticación y operación antes del análisis
Soportar múltiples interfaces	Usar el mismo núcleo desde CLI, TUI, API REST, MCP y extensión VS Code
Exportar resultados reproducibles	Generar reportes en formatos estructurados como JSON y Markdown
Integrar IA de forma opcional	Producir interpretaciones separadas de los datos factuales y tolerar fallos del proveedor
Automatizar calidad y distribución	Validar lint, formato, tipos, pruebas y publicación de artefactos por plataforma

2. Riesgos

Los riesgos identificados se relacionan con la diversidad de motores, la seguridad de credenciales, la interpretación de métricas y la adopción por usuarios con distintos niveles de experiencia.

Riesgo	Impacto	Probabilidad	Tratamiento
Métricas incompatibles entre motores	Alto	Alta	Documentar semántica por motor y evitar un score universal
Motores sin <code>EXPLAIN</code> formal o con datos parciales	Medio	Media	Reportar únicamente la evidencia disponible y conservar el plan original
Exposición de credenciales en archivos, errores o logs	Alto	Media	Cifrado local, <code>SecretStr</code> , sanitización y API sin persistencia de secretos
Dependencias opcionales difíciles de compilar	Medio	Media	Uso de <code>uv.lock</code> , documentación de instalación y binarios por plataforma
Fallos de servicios Docker durante integración	Medio	Media	Separar pruebas unitarias de integración y proveer <code>make up</code> , <code>make health</code> y <code>make down</code>
Interpretaciones incorrectas generadas por IA	Alto	Media	Mantener IA como sección opcional, separada y no factual
Complejidad de uso para usuarios iniciales	Medio	Media	Ofrecer TUI, comandos guiados, extensión VS Code y documentación de uso
Cambios futuros en APIs de drivers o motores	Medio	Media	Encapsular diferencias dentro de adaptadores y ampliar pruebas por contrato

3. Análisis de la Situación Actual

3.1. Planteamiento del problema

Los desarrolladores, estudiantes y administradores de bases de datos que trabajan con más de un motor deben interpretar planes y métricas heterogéneas. PostgreSQL, MySQL, SQLite, SQL Server, CockroachDB, YugabyteDB, MongoDB, Redis, Elasticsearch, DynamoDB, Cassandra, Neo4j e InfluxDB exponen mecanismos de análisis distintos, con salidas que pueden ser JSON, texto, tablas, documentos, árboles, registros de operaciones lentas o estadísticas específicas.

Esta diversidad produce los siguientes problemas:

1. El usuario debe aprender comandos particulares para cada motor.
2. La evidencia queda dispersa entre herramientas, consolas y formatos.
3. Las métricas con nombres similares pueden tener significados diferentes.
4. Se incrementa el riesgo de comparar valores que no son equivalentes.
5. Las credenciales se replican en scripts o herramientas sin un flujo común.
6. Las recomendaciones automáticas pueden confundirse con hechos si no se identifica su origen.

Query Analyzer responde a esta situación mediante una interfaz común que obtiene la evidencia real de cada motor, la normaliza parcialmente para su presentación y conserva el plan original para trazabilidad.

3.2. Consideraciones de hardware y software

El proyecto está diseñado para ejecutarse en equipos de desarrollo convencionales, sin requerir infraestructura dedicada. El usuario final puede trabajar con un binario, con la instalación desde gestores de paquetes o desde el código fuente.

Hardware considerado

Recurso	Requerimiento recomendado	Uso
Computadora personal	8 GB RAM o superior	Ejecución de CLI, TUI, API local y extensión
Procesador	x64 o ARM64 moderno	Análisis local y ejecución de binarios
Almacenamiento	1 GB libre para entorno y dependencias	Código, perfiles, historial y artefactos
Red local o internet	Según motor analizado	Conexión a bases de datos locales o remotas
Docker opcional	Recursos variables por servicio	Pruebas de integración con motores reales

Software considerado

Capa	Tecnología
Lenguaje	Python 3.14+
Gestión de dependencias	uv
Modelos y validación	Pydantic
CLI	Typer y Rich
TUI	Textual
API	FastAPI y Uvicorn
MCP	Servidor local por stdio
Extensión	TypeScript y VS Code API
Seguridad	cryptography y SecretStr
Pruebas	pytest, pytest-cov y pruebas de integración
Calidad	Ruff y mypy
Infraestructura local	Docker Compose
Distribución	PyInstaller, JReleaser, VSIX, Homebrew, Scoop y Snap

La tecnología actual es alcanzable para el equipo porque se basa en herramientas de código abierto y flujos reproducibles mediante uv. Para uso académico no se requiere dominio propio ni servidores comerciales; GitHub, GitHub Actions y GitHub Pages cubren el control de versiones, automatización y publicación de evidencias.

4. Estudio de Factibilidad

El estudio de factibilidad evalúa si Query Analyzer puede desarrollarse, operarse, mantenerse y justificarse dentro del contexto académico del curso. La evaluación fue preparada por el equipo de desarrollo a partir del alcance implementado, la arquitectura, las pruebas, los costos estimados y los riesgos identificados. La revisión queda asociada al docente del curso como aprobador académico del documento.

4.1. Factibilidad Técnica

La factibilidad técnica es favorable. El proyecto cuenta con recursos tecnológicos disponibles y aplicables a las necesidades del sistema:

- Python permite implementar el núcleo, adaptadores, CLI, TUI, API y MCP en un mismo ecosistema.
- Pydantic define contratos claros para configuración, reportes y esquemas REST.
- Typer, Rich y Textual cubren las experiencias de terminal requeridas.
- FastAPI permite exponer el mismo núcleo mediante HTTP local.
- Docker Compose facilita validar motores reales durante pruebas de integración.
- VS Code API permite integrar la herramienta en el editor usado por desarrolladores.
- GitHub Actions automatiza pruebas, reportes, releases y publicación de documentación.

La arquitectura por adaptadores reduce el impacto de la heterogeneidad. Cada motor implementa su propia forma de conexión, diagnóstico, ejecución de `EXPLAIN` o mecanismo equivalente y extracción de métricas. Las interfaces superiores consumen `QueryAnalysisReport`, evitando dependencias directas con drivers concretos.

Recursos técnicos existentes

Recurso	Estado	Evaluación
Código fuente modular	Disponible	Suficiente para mantener y ampliar adaptadores
Entorno reproducible con <code>uv.lock</code>	Disponible	Reduce errores por versiones de dependencias
Docker Compose para integración	Disponible	Adecuado para pruebas con servicios reales
Pruebas unitarias y de contrato	Disponible	Permiten verificar el núcleo sin Docker
API REST local	Disponible	Facilita integración con clientes externos
Servidor MCP	Disponible	Permite uso por agentes compatibles
Extensión VS Code	Disponible	Mejora adopción para usuarios del editor
Documentación y GitHub Pages	Disponible	Centraliza evidencias y guías

Motores contemplados

Categoría	Motores
SQL y NewSQL	PostgreSQL, MySQL, SQLite, Microsoft SQL Server, CockroachDB y YugabyteDB
NoSQL	MongoDB, Redis, DynamoDB, Cassandra y Elasticsearch
Grafos	Neo4j
Series de tiempo	InfluxDB

El proyecto no depende de hardware especializado. Para desarrollo e integración se requiere una computadora capaz de ejecutar Python y, opcionalmente, contenedores Docker. Para uso final, la ejecución por binario reduce la necesidad de instalar dependencias de desarrollo.

4.2. Factibilidad Económica

La factibilidad económica es favorable porque el proyecto utiliza principalmente herramientas libres o gratuitas para uso académico. No requiere licencias comerciales para desarrollar, probar, documentar o distribuir la solución. Los costos se concentran en tiempo del equipo, energía, conectividad y gastos menores.

4.2.1. Costos generales

Los costos generales agrupan materiales de apoyo y gastos menores necesarios para la preparación del proyecto.

Concepto	Cantidad	Costo unitario	Total
Material de oficina y apuntes	1	S/ 40.00	S/ 40.00
Contingencia y gastos menores	1	S/ 191.50	S/ 191.50
Total costos generales			S/ 231.50

4.2.2. Costos operativos durante el desarrollo

Los costos operativos corresponden a servicios necesarios para trabajar durante el periodo del proyecto.

Concepto	Meses	Costo mensual	Total
Servicio de internet	4	S/ 40.00	S/ 160.00
Energía eléctrica	4	S/ 25.00	S/ 100.00
Total operativo			S/ 260.00

4.2.3. Costos del ambiente

El ambiente técnico no requiere inversión inicial en licencias o infraestructura comercial. Los servicios empleados tienen alternativas gratuitas suficientes para el alcance académico.

Recurso	Modalidad	Costo
Python, uv y librerías	Código abierto	S/ 0.00
Docker para uso académico	Gratuito	S/ 0.00
GitHub y GitHub Actions	Plan gratuito	S/ 0.00
GitHub Pages	Plan gratuito	S/ 0.00
Visual Studio Code	Gratuito	S/ 0.00
PyPI, GitHub Releases y artefactos	Gratuito para el alcance académico	S/ 0.00
Total de ambiente		S/ 0.00

4.2.4. Costos de personal

El costo de personal se valoriza académicamente para reflejar el esfuerzo de análisis, diseño, implementación, pruebas, documentación y distribución. No representa un pago efectuado.

Rol / actividad	Responsables	Horas estimadas	Valor hora	Total
Análisis, diseño y arquitectura	ACV, FYG	70.0	S/ 15.00	S/ 1,050.00
Implementación de adaptadores	ACV, FYG	170.0	S/ 15.00	S/ 2,550.00
CLI, TUI, API e integraciones	ACV, FYG	127.5	S/ 15.00	S/ 1,912.50
Pruebas, distribución y documentación	ACV, FYG	73.5	S/ 15.00	S/ 1,102.50
Total de personal		441.0		S/ 6,615.00

La organización de trabajo se divide en dos perfiles principales: desarrollo del núcleo y adaptadores, e integración de interfaces, pruebas y documentación. El horario fue flexible y académico, priorizando entregables por iteración y validaciones automatizadas.

4.2.5. Costos totales del desarrollo del sistema

Categoría	Monto
Costos generales	S/ 231.50
Costos operativos	S/ 260.00
Costos del ambiente	S/ 0.00
Costos valorizados de personal	S/ 6,615.00
Costo total estimado	S/ 7,106.50

La forma de pago real no aplica por tratarse de un proyecto académico. Para análisis de factibilidad, el monto se considera como inversión valorizada del esfuerzo y recursos utilizados.

4.3. Factibilidad Operativa

La factibilidad operativa es positiva porque el sistema puede ser usado por los perfiles previstos sin requerir infraestructura compleja. El usuario puede instalar un binario, crear o seleccionar un perfil, diagnosticar la conexión, ejecutar un análisis y exportar el resultado.

Beneficios operativos

- Reduce el número de comandos específicos que el usuario debe recordar por motor.
- Centraliza perfiles y diagnósticos de conexión.
- Presenta planes y métricas en un reporte uniforme.
- Conserva datos originales para trazabilidad.
- Permite uso por CLI, TUI, API REST, MCP y VS Code.
- No requiere almacenar credenciales en un servicio externo.
- Permite trabajar sin IA; si la IA falla, el reporte factual sigue siendo válido.

Interesados

Interesado	Interés principal
Estudiantes de bases de datos	Aprender planes de ejecución con evidencia reproducible
Docente del curso	Evaluar aplicación de conceptos de rendimiento y arquitectura
Desarrolladores	Diagnosticar consultas durante desarrollo
Administradores de bases de datos	Obtener reportes rápidos y comparables por contexto
Usuarios de VS Code	Analizar consultas desde el editor
Agentes compatibles con MCP	Invocar análisis estructurado desde herramientas de asistencia

4.4. Factibilidad Legal

La factibilidad legal es viable. El proyecto utiliza dependencias y herramientas de código abierto o gratuitas para el alcance académico. El sistema no necesita almacenar datos en servidores externos y está diseñado para proteger credenciales locales.

Se consideran los siguientes aspectos:

- Cumplimiento de licencias de dependencias de código abierto.
- No persistencia de credenciales recibidas por la API REST.
- Cifrado de contraseñas en perfiles locales.
- Sanitización de URIs, tokens, claves y cabeceras sensibles.
- Ejecución local por defecto de la API en `127.0.0.1`.
- Consideración de la Ley N.º 29733 de Protección de Datos Personales cuando las consultas o planes contengan información asociada a personas.

No se identifican restricciones legales que impidan el desarrollo o uso académico del proyecto, siempre que se respeten licencias, privacidad y manejo responsable de datos.

4.5. Factibilidad Social

La factibilidad social es favorable porque el proyecto contribuye al aprendizaje y a la democratización de herramientas de diagnóstico. Query Analyzer permite que estudiantes y desarrolladores sin experiencia profunda en cada motor puedan observar planes reales, entender diferencias y justificar decisiones técnicas.

El proyecto promueve prácticas de ética técnica al evitar presentar recomendaciones automáticas como hechos. También reduce la dependencia de herramientas comerciales y favorece el uso de software abierto en entornos académicos.

4.6. Factibilidad Ambiental

La factibilidad ambiental es aceptable. El impacto directo del proyecto es reducido porque se ejecuta localmente y no requiere servidores permanentes para operar. La mayor parte del consumo se concentra en el equipo de desarrollo y, durante pruebas, en servicios Docker temporales.

El sistema puede aportar beneficios indirectos al facilitar la optimización de consultas: una consulta mejorada puede reducir uso de CPU, memoria, disco, red y recursos cloud. Para mantener bajo el impacto ambiental, se recomienda apagar servicios de prueba con `make down` cuando no se utilicen y ejecutar integraciones completas solo cuando aporten evidencia necesaria.

5. Análisis Financiero

El análisis financiero evalúa la relación entre la inversión valorizada y los beneficios esperados. Al tratarse de un proyecto académico, no se proyectan ingresos por venta; la evaluación se enfoca en beneficios tangibles e intangibles derivados del ahorro de tiempo, la reutilización del sistema y la reducción de errores de diagnóstico.

5.1. Justificación de la inversión

La inversión estimada de **S/ 7,106.50** se justifica por el valor funcional obtenido: una herramienta completa para analizar consultas en múltiples motores, documentada, probada y distribuible.

5.1.1. Beneficios del proyecto

Beneficios tangibles

Beneficio	Descripción
Reducción de tiempo de diagnóstico	El usuario ejecuta un flujo común en lugar de consultar manualmente cada motor
Menor costo de licencias	El stack principal es abierto o gratuito para el alcance académico
Reutilización de perfiles	Las conexiones se gestionan desde una herramienta común
Automatización de evidencia	Los reportes exportables reducen preparación manual de documentación
Distribución multiplataforma	Los binarios y paquetes disminuyen tiempo de instalación del usuario final

Beneficios intangibles

Beneficio	Descripción
Mejora del aprendizaje	Facilita comprender planes de ejecución reales
Mayor confiabilidad	Evita inventar métricas ausentes y conserva planes originales
Mejor toma de decisiones	Proporciona evidencia verificable antes de optimizar
Seguridad operativa	Reduce exposición accidental de credenciales
Interoperabilidad	Permite uso desde CLI, TUI, API, MCP y VS Code
Ventaja académica	Demuestra arquitectura, calidad, pruebas y distribución moderna

Para el análisis de beneficio/costo se valoriza conservadoramente el beneficio del proyecto como **S/ 8,800.00**, considerando el ahorro de horas de diagnóstico, documentación, aprendizaje y reutilización durante el ciclo académico.

5.2. Criterios de inversión

5.2.1. Relación Beneficio/Costo (B/C)

La relación Beneficio/Costo se calcula dividiendo los beneficios valorizados entre el costo total estimado.

Concepto	Monto
Beneficio valorizado estimado	S/ 8,800.00
Costo total estimado	S/ 7,106.50
Relación B/C	1.24

Como la relación B/C es mayor a 1, el proyecto se considera económicamente aceptable para el contexto académico.

5.2.2. Valor Actual Neto (VAN)

Para una evaluación simplificada se considera un costo de oportunidad de capital del 10% anual. El beneficio valorizado se descuenta a valor presente:

Concepto	Monto
Beneficio valorizado	S/ 8,800.00
Factor de descuento al 10%	0.9091
Beneficio a valor presente	S/ 8,000.00
Inversión estimada	S/ 7,106.50
VAN estimado	S/ 893.50

Como el VAN estimado es mayor que cero, el proyecto se acepta financieramente bajo esta estimación académica.

5.2.3. Tasa Interna de Retorno (TIR)

En un escenario simplificado de un periodo, la TIR aproximada se obtiene comparando el beneficio valorizado con la inversión:

Concepto	Valor
Beneficio valorizado	S/ 8,800.00

Concepto	Valor
Inversión estimada	S/ 7,106.50
TIR aproximada	23.83%
Costo de oportunidad de capital	10.00%

La TIR aproximada supera el costo de oportunidad de capital, por lo que el proyecto se considera financieramente viable dentro de los supuestos académicos utilizados.

6. Conclusiones

1. Query Analyzer es técnicamente factible porque se apoya en una arquitectura modular, tecnologías abiertas y pruebas automatizadas que permiten mantener múltiples motores bajo un contrato común.
2. La factibilidad económica es favorable: el proyecto no requiere licencias comerciales y su costo principal corresponde al esfuerzo valorizado del equipo.
3. La operación del sistema es viable porque puede utilizarse desde CLI, TUI, API REST, MCP y VS Code, con perfiles locales y diagnósticos progresivos.
4. La factibilidad legal es aceptable siempre que se respeten licencias, privacidad, protección de datos y manejo seguro de credenciales.
5. La factibilidad social es positiva porque facilita el aprendizaje y uso de evidencia real de rendimiento en entornos académicos y de desarrollo.
6. La factibilidad ambiental no presenta restricciones significativas, ya que el sistema se ejecuta localmente y puede contribuir indirectamente a reducir consumo de recursos mediante optimización de consultas.
7. El análisis financiero muestra una relación B/C de 1.24, VAN positivo de S/ 893.50 y TIR aproximada de 23.83%, por lo que el proyecto resulta viable bajo los supuestos académicos planteados.
8. La decisión de no generar puntuaciones universales ni mezclar IA con evidencia factual incrementa la transparencia del sistema y reduce riesgos de interpretación.
9. El proyecto es viable y factible para el contexto de Base de Datos II, tanto como producto académico como herramienta práctica para diagnóstico de consultas.